

Document Individuel — Alexis Ballenghien

PARTIE A — MA CONTRIBUTION AU PROJET

Les écrans et fonctionnalités développés

En tant qu'étudiant travaillant en solo, j'ai développé l'intégralité de l'application **CineMatch**, du concept à la mise en production. Cela inclut :

Navigation et authentification :

- Stack de navigation Expo Router avec authentification Supabase
- Écrans de login/signup avec gestion de session sécurisée
- Context AuthContext pour la persistance JWT

Écrans principaux (3 onglets) :

- **Explore** : Écran de swipe avec gestures PanResponder, affichage film 1 par 1, gestion like/reject persistée
- **Matches** : Grille des films aimés (2 colonnes), tri par note, pull-to-refresh
- **History** : Historique des swipes avec filtres (Tous/Aimés/Rejetés), dates en français

Fonctionnalités additionnelles :

- **Recherche** : Barre de recherche avec debounce 500ms, pagination, 4 empty states
- **Détail film** : Synopsis, genres, casting (5 premiers acteurs), lien YouTube trailer
- **Filtres par genre (Bonus)** : Dropdown horizontal, intégration TMDb getMoviesByGenre
- **Animations Reanimated (Bonus)** : Rotation 3D (-15° à +15°), spring bounce-back, feedback opacity

Qualité technique :

- Structure src/ organisée (components/, hooks/, services/, contexts/)
- 8 composants réutilisables (MovieCard, MatchMovieCard, SwipeCard, GenreFilter, etc.)
- Services séparés : tmdb.ts, database.ts, auth.ts, supabase.ts
- TypeScript strict, ESLint configuré
- 20+ commits Git réguliers avec messages explicites

Difficultés techniques rencontrées

1. Vérification d'email Supabase

- **Problème** : Supabase imposait une vérification d'email avant d'autoriser l'accès aux données. Cela bloquait le flux utilisateur après inscription.
- **Impact** : Les utilisateurs ne pouvaient pas swiper les films immédiatement après signup.
- **Résolution** : J'ai désactivé la vérification d'email obligatoire dans les paramètres Supabase (Email Confirmations = OFF). Cela permet aux utilisateurs de se connecter immédiatement après inscription. J'ai ensuite supprimé l'écran verify-email.tsx qui devenait inutile.

2. Création de profil et RLS (Row-Level Security)

- **Problème** : Après signup, il fallait créer automatiquement un profil utilisateur dans la table profiles. Les politiques RLS empêchaient les inserts non autorisés.
- **Impact** : Certains utilisateurs n'avaient pas de profil, causant des erreurs lors des requêtes aux tables liked_movies et swipe_history.
- **Résolution** :
 - J'ai implémenté une vérification dans authService.createProfile() appelée après signup
 - Configuration correcte des RLS politiques pour autoriser les inserts
 - Catch des erreurs de duplication avec try/catch

3. Gestion du débounce de recherche

- **Problème** : Sans debounce, chaque caractère tapé déclenchait un appel API à TMDB
- **Résolution** : Implémentation d'un hook useMovieSearch avec setTimeout, délai 500ms

4. Pagination et restauration de progression

- **Problème** : À chaque rechargement, l'utilisateur recommençait au film 1
- **Résolution** : Calcul du swipeCount depuis swipe_history, chargement des pages jusqu'à targetPage, restauration avec setCurrentIndex

5. TS2345 et erreurs de type TypeScript

- **Problème** : Multiples erreurs de type : TS2345, TS2322, TS2769, TS2694
- **Résolution** : Correction des imports (SharedValue au lieu de Animated.Shared), ajustement des RefObject types, cast when necessary

Ce que j'aurais fait différemment avec plus de temps

1. **Tests unitaires et E2E** : Jest + Detox pour tester les flux critiques
2. **State management** : Redux ou Zustand au lieu de Context pour meilleure performance
3. **Notifications push** : Alerter les utilisateurs des matches
4. **Real-time** : Supabase Realtime pour collaborative viewing
5. **Haptic feedback** : Vibrations sur les swipes
6. **Onboarding** : Tutoriel interactif pour nouveaux utilisateurs

PARTIE B — MA UTILISATION DE L'IA

Outils IA utilisés

- **GitHub Copilot** : Autocomplétion, génération de code, refactoring
- **Processus** : PLAN (avec agent autoskill) → IMPLEMENTATION (avec Copilot) → REVIEW + ADAPT

Exemples concrets de prompts utilisés

Prompt 1 : Création du hook useMovieStack

```
"Create a custom React hook called useMovieStack that manages  
a stack of movies with pagination, filtering by genre,  
and restoration of user progress on reload. Include refs  
for movies, currentIndex, and page."
```

Résultat : Skeleton du hook avec useState/useRef, fonction loadMovies avec filtrage, effets pour pagination

Amélioration manuelle : Ajout boucle de retry si page 1 vide après filtrage, fixage dépendances useCallback

Prompt 2 : Animations Reanimated 3D

```
"Generate React Native Reanimated animation code for a card:  
- Rotate X axis from -15° to +15° based on panX value  
- Spring back to center when released (withSpring)  
- Show green overlay on right, red on left with opacity"
```

Résultat : useSharedValue, useAnimatedStyle, interpolate, spring config **Amélioration** : Changé Animated.Shared en SharedValue, ajout Extrapolate.CLAMP, calibrage sensibilité

Prompt 3 : Fix TS2345 type errors

```
"Fix TypeScript errors:  
- TS2345: Argument '(prev) => prev + 1' not assignable to 'number'  
- TS2322: RefObject type mismatch"
```

Résultat : Changement setPage pattern, ajout | null à types generiques **Apprentissage** : Comprendre les types génériques plutôt que les ignorer

Prompt 4 : Pull to refresh pour empty state

```
"Add pull-to-refresh to empty state using RefreshControl inside ScrollView.  
Use isRefreshing state and handleRefresh callback that calls loadMovies(1)"
```

Résultat : ScrollView avec RefreshControl, useState, callback setup **Amélioration** : Message d'aide visible, GenreFilter reste visible

Prompt 5 : MatchMovieCard pour grille

```
"Create MatchMovieCard component for liked movies grid (2 columns).  
Show: poster (75%), title, rating. Similar to MovieCard but optimized."
```

Résultat : Structure MovieCard adaptée, flex ratios corrects **Amélioration** : Font sizes optimisés, séparation de MovieCard pour éviter coupling

Ce que l'IA a bien fait

1. ☐ **Boilerplate rapide** : Structure complète en secondes
2. ☐ **Refactoring** : Proposer des noms meilleurs, patterns React
3. ☐ **Pattern recognition** : Reconnaître les patterns (debounce, animations)
4. ☐ **TypeScript** : Inférer les types génériques correctement
5. ☐ **Documentation** : Générer des JSDoc comments utiles

Ce que l'IA a mal fait

1. ☐ **Logique métier complexe** : Besoin de guidance pour "si page 1 vide alors essayer 2-5"
2. ☐ **Imports** : Suggestion de Animated.Shared au lieu de SharedValue
3. ☐ **Nommage** : Génériques (handlePress) au lieu de contextuels
4. ☐ **Performance** : Oublie useCallback, dépendances douteuses
5. ☐ **Sécurité** : Ne propose pas SecureStore spontanément

Ma réflexion personnelle : Comment l'IA a changé ma façon de coder

1. Planification avant code

- Avant : J'allais droit au code, souvent inefficace
- Après : PLAN → IMPLEMENT → REVIEW avec prompts détaillés
- Résultat : **Moins de refactoring, code plus prédictible**

2. Styles modernes et TypeScript

- Avant : Peur des types complexes (generics, conditional types)
- Après : Copilot me montre des patterns TypeScript avancés
- Résultat : **TypeScript assurée, zéro any inutile**

3. Documentation technique

- Avant : Doc écrite après, vague
- Après : JSDoc au moment du code via IA
- Résultat : **PROJECT_CONTEXT.md et HISTORY.md maintenus**

4. Historisation et traçabilité

- Avant : Commits "fix stuff" ou "wip"
- Après : Copilot m'oblige à penser en "features cohérentes"
- Résultat : **20+ commits structurés, l'historique raconte la story**

5. Débogage plus efficace

- Avant : Stack trace → Google → copy/paste aveugle
- Après : Stack trace → prompt précis à Copilot + compréhension
- Résultat : **Erreur comprise, pas juste résolue**

Conclusion : L'IA n'a pas écrit mon app pour moi. Elle m'a permis de **penser plus clairement, d'écrire plus vite, et de documenter mieux**. Je reste le pilote — l'IA c'est le copilote intelligent qui m'aide à clarifier mes idées avant le code.

Document rédigé par : Alexis Ballenghien

Date : 5 mai 2026

Projet : CineMatch — Application de swipe films React Native / Expo

Statut : Solo, toutes fonctionnalités